PRODUCT AND OPERATIONS GUIDE

StockPulse

Paper-first trading dashboard, Kronos forecasts, WallStreetBets research, and a look-ahead-safe historical backtester.

Version: unreleased personal build

Date: 19 August 2026

Local site: http://localhost:5173

LAN: http://192.168.86.197:5173

> Not investment advice. Forecasts are research outputs. Broker acceptance does not guarantee execution. Use paper mode until you understand the risk controls.

1. Overview

StockPulse is a personal trading workstation. It shows paper (or live) market data, Kronos probabilistic forecasts, news and public sentiment, a background scan of predicted movers, and your open broker positions. Orders are manual only. Forecasts never submit trades.

The repository also contains Kronos itself (tokenizer, predictor, fine-tuning examples) and a separate production-grade backtester in kronos_backtest/.

2. Architecture

Layer	Path	Role
Dashboard	frontend/	React + TypeScript + Vite. Proxies /api to the backend.
API	backend/	FastAPI on loopback :8000. Alpaca, Finnhub, Kronos.
Model	model/	Kronos predictor/tokenizer. Loaded on first forecast.
Backtester	kronos_backtest/	Look-ahead-safe engine, costs, walk-forward, reports.
Legacy demo	webui/	Unchanged Flask demo. Not required for StockPulse.

The backend binds to 127.0.0.1:8000. LAN clients reach it only through the frontend proxy on port 5173. Do not expose the API directly on the network unless you intend to.

Request path

```
Browser (localhost or LAN)
  -> Vite preview :5173
       -> static dashboard
       -> /api/v1/* proxy -> FastAPI :8000 (loopback)
            -> Alpaca / Finnhub / Kronos
```

3. Local hosting

StockPulse is meant to run as a normal Windows website, not inside a Cursor terminal. Two Task Scheduler tasks start at Windows logon:

Task	Script	Listens
KronosWeb	scripts/start-kronos-web.ps1	0.0.0.0:5173 Vite preview of the production build
KronosAPI	scripts/start-kronos-api.ps1	127.0.0.1:8000 uvicorn

Open http://localhost:5173 in Edge or Chrome. Other devices on the same Wi-Fi can use http://192.168.86.197:5173 if the PC firewall allows inbound TCP 5173.

Manual start

```
powershell -ExecutionPolicy Bypass -File scripts/publish-kronos-lan.ps1 -SkipChecks
```

That script builds the frontend if needed, starts both processes detached, and prints the local and LAN URLs. After application code changes, run backend and frontend checks first. Documentation-only changes do not need republishing.

Logs

- runtime-logs/web.log website (scheduled task)
- backend/logs/api.log API (scheduled task)
- runtime-logs/backend.*.log API when started via the LAN publish script

4. Dashboard

- Search: symbol or company name; results stay in a scrollable list.
- Market light: US session pre-market, regular, after hours, or closed (America/New_York).
- Quote card: last, change, OHLC, volume, market cap, P/E, EPS, dividend yield.
- Chart: historical OHLC plus Kronos path (or ensemble Forecast). Short vs long horizon.
- Sentiment: Public (Finnhub news) and Investors (Kronos trend). A dimmed investor badge means no reliable out-of-sample edge.
- News: merged Finnhub + Alpaca coverage for the selected symbol.
- Portfolio / ticket: paper or live, equity or single-leg option, review-before-send.
- Movers: top 50 predicted gainers and losers from a background Kronos scan.
- Open positions: full-width holdings table under the movers scan. Click a symbol to load it.

Partial-data banners appear when quote data loaded but chart or forecast did not (including brief forecast throttling). Forecast retries and in-flight request coalescing reduce false unavailable states when switching symbols.

5. Forecasts and movers

Kronos is loaded on the first forecast, not at API startup. Device is auto unless overridden (cpu, cuda:0, MPS). Context is capped at 512. Future timestamps skip weekends and stay inside the US regular session. That is market-aware, not a full holiday calendar.

Preset	Bars	Context	Horizon
short	5-minute	256	12 bars
long	daily	256	20 bars

The movers scan ranks today's active names by absolute Kronos forecast change after a spread/slippage haircut. It uses a separate rate-limit bucket from per-ticker forecasts (defaults: 10 scan requests/min vs 30 ticker forecasts/min) so a universe scan does not starve the chart. POST /forecast/movers starts a background scan.

Poll GET /forecast/movers/status for progressive top-50 gainers and losers.

6. Trading and safety

- Default mode is paper. Live is disabled until ALLOW_LIVE_TRADING=true on the server.
- Enabling live in the UI requires typing LIVE. Live orders also send that confirmation token.
- Every order opens a review dialog. There is no scheduler or signal executor.
- Equity sells cannot exceed the long position unless ALLOW_SHORT_SELLING=true.
- Option sell_to_open requires ALLOW_UNCOVERED_OPTIONS=true.
- Risk preview can block buys (position size, spread, daily-loss halt).
- Cancel open orders from Activity. Replace is available on the API.

Paper smoke checklist

1. Configure only paper Alpaca keys (and optionally Finnhub).
2. Keep ALLOW_LIVE_TRADING=false.
3. Confirm /health and /config/status show no secrets.
4. Search AAPL, open overview, load daily bars.
5. Inspect paper account, positions, and orders.
6. View an option chain without submitting.
7. Place a one-share or low-notional paper order; confirm in Alpaca.
8. Cancel an open paper order from the UI or API.
9. Run a short Kronos forecast (first call downloads weights).
10. Confirm a live request returns 403 while live is disabled.

7. API reference

All routes use the prefix /api/v1. Interactive docs: http://127.0.0.1:8000/docs (loopback only). Responses include X-Request-ID. Missing providers return generic 503; upstream failures return generic 502. If API_KEY is set, every route except /health requires header X-API-Key.

Method	Path	Purpose
GET	/health	Liveness
GET	/ready	Service presence
GET	/config/status	Safe config flags (no secrets)
GET	/market/clock	Session / next open-close
GET	/symbols/search	Symbol lookup
GET	/stocks/{symbol}/overview	Quote, news, fundamentals, sentiment
GET	/stocks/{symbol}/bars	OHLCV
POST	/forecast	Kronos or ensemble Forecast (engine)
POST	/forecast/movers	Start movers scan
GET	/forecast/movers/status	Scan progress and rankings

Method	Path	Purpose
GET	/account, /positions, /orders	Broker state (mode=paper|live)
GET	/options/contracts, /options/chain	Option chain
POST	/orders/preview	Risk preview
POST	/orders/equity, /orders/option	Submit order
DELETE/PATCH	/orders/{id}	Cancel / replace

Example paper equity order

```
{
  "mode": "paper",
  "symbol": "AAPL",
  "side": "buy",
  "type": "limit",
  "time_in_force": "day",
  "qty": 1,
  "limit_price": 190.00
}
```

8. Configuration

Copy backend/.env.example to backend/.env (gitignored). Important variables:

Variable	Notes
ALPACA_PAPER_KEY / _SECRET	Paper trading and default market data
ALPACA_LIVE_KEY / _SECRET	Live trading only when enabled
ALPACA_DATA_FEED	iex (sip only with a SIP subscription)
FINNHUB_API_KEY	Fundamentals, company news, public sentiment
ALLOW_LIVE_TRADING	false
API_KEY	Empty = no API auth (local default)
CORS_ORIGIN	http://localhost:5173 plus LAN origins as needed
FORECAST_RATE_LIMIT_PER_MINUTE	30
FORECAST_SCAN_RATE_LIMIT_PER_MINUTE	10
KRONOS_MODEL_ID	NeoQuasar/Kronos-small

Frontend optional frontend/.env.local: VITE_API_BASE_URL (default same-origin /api/v1) and VITE_API_KEY if the backend requires a key. A key in the browser is not a substitute for firewall and CORS controls.

9. Historical backtester

kronos_backtest is the supported engine. examples/run_backtest_kronos.py and finetune/qlib_test.py are demo-grade only.

Event loop (no look-ahead)

```
For each bar T:
  1. OHLCV for T is known
  2. Pending orders fill at this bar's OPEN if delay has elapsed
     (never this bar's close, never the signal bar's close)
  3. Mark portfolio to T close
  4. Context = history with timestamps <= T
  5. Predictor sees only that context (LookAheadBiasError otherwise)
  6. Strategy emits BUY / SELL / HOLD after estimated costs
  7. Risk manager sizes or rejects
  8. Order is queued; it cannot fill until a later bar
```

Default: signal at T, fill at T+1 open, plus spread, slippage, and fees. If bid/ask columns are missing, a synthetic spread is applied around the next-bar open.

```
pip install -r requirements.txt
python -m kronos_backtest --config configs/backtest.yaml --predictor dummy --output backtest_results
pytest tests/backtest -q
```

Use --predictor kronos to wrap the real model (downloads weights). Outputs: summary.json, metrics.json, trades.csv, equity/drawdown CSVs, and report.html. Limitations: no market-impact model beyond proportional slippage; only MARKET fills; default loop is one symbol per run; fine-tune quality depends on the callback you supply (test rows are never passed to fit).

10. Development and tests

Use separate virtualenvs for the core model and the StockPulse backend when dependency sets conflict. Application CI uses Python 3.12 and Node 22.

```
cd backend && pytest -q
cd frontend && npm run lint && npm run typecheck && npm test && npm run build
pytest -q tests/backtest
```

Backend tests use fakes and HTTP mock transports. They never call Alpaca, Finnhub, Hugging Face, or place orders. Optional Docker: docker compose up --build from the repo root.

11. Security and licensing

- Never commit .env, credentials, account IDs, or model weights.
- Report vulnerabilities privately via GitHub Security, not a public issue.
- Financial loss, forecast accuracy, and provider outages are not software vulnerabilities.
- Source is MIT unless a file says otherwise. Hugging Face weights, broker data, and third-party packages

keep their own terms.

Related files: SECURITY.md, docs/LICENSING.md, docs/DEVELOPMENT.md, docs/BACKTEST.md, backend/README.md, frontend/README.md, TRADING_APP.md.

End of StockPulse documentation - 19 August 2026. This guide describes the personal StockPulse build in this repository and is not a broker disclosure or offer to transact.